基于执行反馈的 LLM 辅助移动机器人任务规划

候选人面试作业

摘要

本文实现了一个运行于 Webots R2025a 与 ROS 2 Humble 的移动机器人任务规划系统。系统将自然语言请求转换为少量、经过验证的高层技能调用，并遵循 Plan → Validate → Execute → Observe → Recover 闭环。LLM 只能在固定白名单中选择技能；验证器在执行前检查每个调用；执行后系统读取结构化结果，并在超时、障碍、窄缝等失败场景下进行有界恢复，必要时把任务级执行反馈发回 LLM 重规划。我们在实现过程中逐渐认识到：LLM 系统的可靠性来自可验证的架构、结构化的执行反馈，以及与实际仿真模型匹配的控制参数，而不是某一段提示词。正常与可恢复任务 5/5 成功，2 个非法计划全部在执行前被拒绝，2 个不可恢复失败（超时、持续障碍）均安全停车；成功任务平均耗时 8.65 s，可测得的最终位置误差平均约 0.081 m。

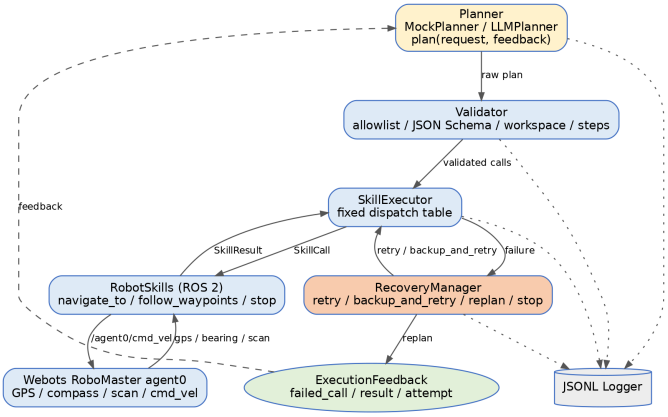


图 1: Plan–Validate–Execute–Observe–Recover 闭环。

1 任务定义与设计目标

目标是让用户通过自然语言指令，例如 “Go to storage, then workbench, and stop” （storage、workbench 等地点是提前在房间内指定的位置），驱动 Agent 完成任务。为了保证安全性和可控性，系统要求 LLM 只在任务层规划：它从固定白名单中选择并组合高层技能，任何调用在执行前必须经过验证器，实际运动由经过测试的 ROS 2 技能层完成；执行失败后系统根据结构化结果进行有界恢复。这样既能利用 LLM 的语言理解能力，又避免 LLM 直接控制机器人带来的风险。

设计目标可以概括为四点：

- **自主性**：用户无需写代码或手动操作，只需自然语言指令；
- **安全性**：白名单、验证器、有界恢复、安全停车；
- **可控性**：每个失败都有明确的恢复动作和次数上限；
- **可复现性**：环境、地图、日志、实验命令完整记录。

项目推进过程中，我们不断修正对 “可靠性” 的理解：一开始以为只要提示词足够严格，LLM 就会给出安全计划；后来发现仅靠提示词无法约束未知技能、越界坐标和控制误差，于是逐步把安全责任从模型行为转移到可检查的架构组件上。这一认识贯穿后文所有设计。

2 系统架构与技能接口

2.1 整体架构

系统由四层组成：Planner、Validator、Skill Executor、ROS 2 技能层，执行结果再进入 Recovery。图 1 给出闭环结构。

- **Planner**：模拟规划器或兼容的 LLM 规划器，输出原始 `SkillCall[]`；
- **Validator**：六层检查，不通过则计划不会到达机器人；
- **Skill Executor**：只把白名单技能名映射到预定义 Python 方法；
- **RobotSkills**：ROS 2 节点，发布 `/agent0/cmd_vel`，读取 GPS、罗盘与 `/scan`；
- **Recovery**：根据 `SkillResult` 选择 retry / backup / replan / stop。

这种分层的意义在于：LLM 可以出错，但只要它无法绕过 Validator，最坏情况也只是计划被拒绝，而不是机器

表 1： 技能接口摘要。

技能	契约
`navigate_to`	参数：x, y, θ；成功：位置误差 $\leq 0.10\,\mathrm{m}$，航向误差 $\leq 0.15\,\mathrm{rad}$；超时：30 s
`follow_waypoints`	参数：航点列表；失败时返回最后到达的航点下标；每个航点超时 30 s
`stop`	发布零速度；成功：已发布零速度；超时：2 s

人收到危险指令。我们没有把安全寄希望于提示词，而是把安全放在 LLM 与执行器之间。

2.2 技能接口

表 1 给出三个技能的契约。注册表是单一事实来源：同一份定义既生成验证器使用的 JSON Schema，也生成 LLM 工具列表，避免接口定义与实际执行漂移。

设计技能接口时，我们刻意避免“返回一个 bool 就结束”。每个技能都返回 `success`、`reason` 和 `elapsed_time_s`，因为上层恢复策略需要知道失败原因，而不仅是失败事实。这一决定是后续执行反馈与有界重规划能够成立的前提。

2.3 具名地点与工作空间

当前具名地点：entrance $(0.0, 3.0)$、storage $(-2.152, -1.363)$、workbench $(4.8, 0.0)$、charging_station $(-2.17, 1.95)$。工作空间为 $x \in [-7, 7]$、$y \in [-4, 7]$。以上坐标与 `agent/locations.py` 中的常量表一致；提示词、验证器工作空间和具名地点均由同一份定义生成，避免接口漂移。开发过程中曾发现提示词把边界硬编码为 $[-1.5, 1.5]^2$，与真实边界不一致，导致真实 LLM 返回空计划；修正后单点与多航点任务均可完成。

3 实现细节

3.1 导航与控制

底盘为全向 RoboMaster，可以同时平移和旋转。罗盘方位角以顺时针为正，ROS 偏航角以逆时针为正，因此使用 $\psi = \mathrm{normalize}(-\mathrm{radians}(\mathrm{bearing}))$ 转换。位置和航向分别使用三组 PID：

$$v_x^w = \mathrm{clamp}(k_p e_x + k_i \int e_x dt + k_d \dot{e}_x, -v_{max}, v_{max}) \quad (1)$$

$$v_y^w = \mathrm{clamp}(k_p e_y + k_i \int e_y dt + k_d \dot{e}_y, -v_{max}, v_{max}) \quad (2)$$

$$\omega = \mathrm{clamp}(k_p^\theta e_\theta + k_i^\theta \int e_\theta dt + k_d^\theta \dot{e}_\theta, -\omega_{max}, \omega_{max}) \quad (3)$$

再把世界系速度旋转到车体系：$v_x^b = \cos\psi\, v_x^w + \sin\psi\, v_y^w$，$v_y^b = -\sin\psi\, v_x^w + \cos\psi\, v_y^w$。当前参数：$v_{max} = 2.25\,\mathrm{m/s}$，$\omega_{max} = 6.0\,\mathrm{rad/s}$。PID 参数为 $k_v = 4.0$、$k_{i,v} = 0.6$、$k_{d,v} = 0.1$，$k_\theta = 5.0$、$k_{i,\theta} = 0.5$、$k_{d,\theta} = 0.05$，积分限幅 0.5，控制频率 20 Hz。开环标定显示：`linear.x=1.0` 约对应 0.10 m/s，`linear.y=1.0` 约对应 0.11 m/s，`angular.z=5.0` 约对应 1.11 rad/s。参考控制器使用的轮半径 0.26 m 与 URDF 的 0.06 m 不一致，因此技能层使用闭环 PID 反馈，而不是按开环时间估算位移；闭环后单点任务最终位置误差约为 0.08 m。

这部分经历了两次明显修正。最初使用纯比例控制，发现到达精度受稳态误差影响；改为 PID 并加入积分限幅后，精度明显改善。随后在调试速度时发现参考控制器的轮半径常量与 URDF 不一致，导致指令速度与实际速度之间关系异常；修正该常量并重新标定后，控制行为才与预期一致。由此得到的经验是：**仿真中的小常量会直接影响控制效果，调参前应先确认模型本身是否正确。**

3.2 地图与 A* 导航

地图有两条生成路径：

- **激光扫描建图**：用 GPS、罗盘和 `/scan` 增量建图；
- **Supervisor 精确建图（默认）**：由 Webots Supervisor 读取场景物体的真实位置、尺寸和旋转，离线生成 PGM/YAML。

A* 在可通行格上规划，并做三点改进：

1. 对离墙近的格子增加代价，使路径偏向通道中间；
2. 离障碍物小于 0.15 m 的格子视为不可通行，避免钻入窄缝；
3. 路径直线简化时同样要求最小离障距离。

Supervisor 地图为 800×640 栅格、分辨率 0.02 m/cell、原点 $[-8, -4]$，覆盖 16.0 m×12.8 m，房间自由区域约 61.0 m^2（未知区域主要是房间外）。早期激光增量地图为 200×200、分辨率 0.02 m/cell、原点 $[-2, -2]$，未知率约 57%，storage 与 workbench 已在图外，A* 无法给出到这些点的路径。在 Supervisor 地图上，以不启用离墙代价与最小离障距离为基线，改进 A* 在 storage→workbench、entrance→workbench、entrance→storage 三条路线上路径长

度平均增加约 4.6%（如 8.49 m→9.04 m），但最小离障距离从约 0.02 m 提高到约 0.25–0.30 m；其中 entrance→storage 长度不变，最小离障距离从 0.028 m 提高到 0.300 m。

建图方法的演化本身说明了“数据源决定算法上限”。激光扫描受机器人轨迹限制，覆盖不全时 A* 只能在残缺地图上规划；改用手动巡逻又容易被障碍卡住。最终改用 Supervisor 直接读取物体几何，几秒内得到全房间地图，A* 才真正发挥出作用。这让我们意识到：**与其反复调整规划器参数，不如先解决地图质量这一上游问题。**

3.3 局部安全：障碍、窄缝与贴墙

除全局 A* 外，技能层在控制周期内执行局部保护：

- **前方障碍**：运动方向锥形区域内最近距离小于 0.15 m 时停车；
- **窄缝检测**：估算前方自由缝隙宽度，小于 0.45 m 时停车；
- **侧向排斥**：左右间隙小于 0.30 m 时产生横向排斥速度；
- **窄道减速**：左右间隙越窄，速度越低，最低为原速度的 25%。

这些参数不是一次性确定的。障碍膨胀太大会堵死通道，太小会贴墙；窄缝检测太敏感会误停车，太宽松会卡住。最终采用“离墙代价 + 最小离障距离 + 窄缝检测 + 侧向排斥 + 窄道减速”的组合策略。经验是：**安全参数不是越大越好，必须在具体场景中权衡并实测。**

3.4 LLM 工具调用与验证

LLM 只接收四类信息：系统提示词、具名地点、工作空间边界、由注册表生成的工具定义。系统提示词明确要求只选择白名单技能，禁止生成代码或 shell 命令。如果未配置 API key 或 API 调用失败，系统回退到确定性的模拟规划器，保证其余部分可测试。真实 LLM 通过 OpenAI 兼容接口调用：`OPENAI_BASE_URL=https://yinli.one/v1`，`OPENAI_MODEL=gpt-5.5`，使用 function calling 返回工具调用；API key 只保存在被 gitignore 的`.env` 中。系统提示词由代码动态生成，核心摘录如下：

```
You are a robot task planner. Select only from the
    provided skills.
Do not generate code or shell commands.
Available named locations:
- entrance: x=0.0, y=3.0, theta=0.0
- storage: x=-2.152, y=-1.363, theta=0.0
...
Workspace bounds: x in [-7.0, 7.0], y in [-4.0,
    7.0].
Use navigate_to for a single target pose and
    follow_waypoints for an
ordered list of targets. If the user explicitly asks
     to stop, include a
stop tool call after the movement tools.
```

这里也遇到一个典型问题：早期提示词把工作空间硬编码为 $[-1.5, 1.5]^2$，与 `locations.py` 的真实边界不一致，真实 LLM 在部分请求下返回空计划。改为从统一的常量表生成提示词与验证器边界后，单点与多航点任务均可成功完成。这说明提示词、工具 Schema 与验证器必须共享同一份事实来源。

验证器包含六层：计划长度、固定白名单、JSON Schema、工作空间、前置条件、重试/重规划限制。执行器只把三个技能名映射到 Python 方法，不存在 LLM 生成代码的执行路径。回头来看，这套验证器比任何提示词工程都更关键：**对 LLM 系统而言，安全必须由可验证的架构保证，而不是靠模型自觉。**

3.5 执行反馈与有界恢复

每个技能返回 `SkillResult`，至少包含 `success`、`reason`、`elapsed_time_s`。失败后 Recovery 根据原因选择：

- `retry`：超时后重试一次；
- `backup_and_retry`：障碍阻塞时后退、旋转并重试一次；
- `replan`：重试预算耗尽后，把任务级 `ExecutionFeedback` 发回 LLM；
- `stop_and_report`：达到恢复上限后停车并报告失败。

`ExecutionFeedback` 不包含原始激光或 GPS 数据，只包含原始请求、失败调用、结构化结果、尝试次数、恢复消息与已完成工作。`--max-retries` 与 `--max-replans` 保证恢复有界。

实现这一部分时我们意识到：**执行反馈的价值取决于它的结构。**如果只告诉 LLM “失败了”，它无法区分超时、障碍和越界；如果给 LLM 原始传感器数据，又会引入新的不确定性和安全隐患。因此我们选择只传递任务级、结构化、有界的信息，让重规划发生在可控范围内。

表 2：关键指标汇总。

指标	结果
正常/可恢复任务	5/5 成功
非法/未知计划	2/2 执行前拒绝
不可恢复失败	2/2 安全停车
成功任务平均耗时	8.65 s
可测平均位置误差	0.081 m
真实 LLM 单点/多航点	4.25 s / 10.23 s
LLM 反馈重规划	链路跑通（A11）

4 实验与结果

实验平台为 Windows 11 + WSL2 Ubuntu 22.04、ROS 2 Humble、Webots R2025a、Python 3.10.12，GPU 为 AMD Radeon RX 9070 XT。表 2 汇总关键指标，详细结果见下文。

- A1/A2：正常单点/多航点导航成功，5.73 s 与 16.44 s；
- A3/A4：space_station 与 fly_to 在执行前被验证器拒绝；
- A5/A7：1 s 超时和持续障碍分别在 3.66 s、3.35 s 内安全停车；
- A6：障碍消失后 backup+retry，6.57 s 到达，误差 0.082 m；
- A8/A9：真实 LLM 单点与多航点成功，4.25 s、10.23 s，误差 0.080 m；
- A10：合成 6° 自由扇区估计缝宽 0.03 m 并触发停车，40° 扇区不误停；
- A11：超时后 LLM 收到 `ExecutionFeedback` 并改去 entrance，新计划通过验证后执行；受 1 s 超时限制最终安全停车，反馈闭环已跑通。

实验说明：正常与可恢复任务 5/5 成功，非法/未知计划 2/2 在执行前拒绝，不可恢复超时/持续障碍 2/2 安全停车。窄缝 A10 目前是合成扫描单元测试；受房间几何限制，还缺少真实窄缝中的长距离通行验证。

5 局限与下一步

现有系统仍有几处局限。首先，WSLg 渲染存在额外开销，影响演示流畅度。其次，参考环境没有 `/agent0/odom`，位置和航向只能依赖 GPS 与罗盘，这限制了更精细的运动估计。第三，参考控制器轮半径常量与 URDF 不一致，我们通过修正常量来规避，但这类外部依赖仍是潜在风险。最后，Supervisor 地图虽然比扫描地图完整，但对个别复杂物体仍采用近似尺寸。真实 LLM 的延迟和输出稳定性也是外部变量：一次重规划调用可能耗时数十秒，并曾因提示词与工作空间不一致而返回空计划。

下一步工作会沿着“让系统更可靠”展开：

- 融合 SLAM 与几何地图，兼顾完整性与真实观测；
- 增加动态障碍规避，处理移动物体而不仅是静态地图；
- 在更多失败场景中系统评估 LLM 重规划，而不仅是 unit-level 触发；
- 为 LLM 调用增加超时、缓存与空计划回退，降低外部 API 波动的影响；
- 提供 Docker/WSL 一键复现环境，降低环境差异带来的风险。

6 结论

本文实现了一个带执行反馈的 LLM 辅助移动机器人任务规划系统，覆盖自然语言规划、技能接口、六层验证、PID 导航、A* 地图导航、局部安全保护与有界恢复。项目过程中我们逐步认识到：**LLM 系统的可靠性不是来自某一段提示词，而是来自可验证的架构、结构化的执行反馈，以及与实际仿真模型匹配的控制与地图。**我们把 LLM 限制在任务级规划，把安全责任交给验证器、技能层与有界恢复，并通过多轮问题定位与修正验证了这条路线。

参考文献

[1] ROS 2 Humble 文档. https://docs.ros.org/en/humble/

[2] Webots R2025a. https://cyberbotics.com/

[3] webots_ros2. https://github.com/cyberbotics/webots_ros2

[4] webots_ros2_robomaster. https://github.com/MohismLab/webots_ros2_robomaster

[5] OpenAI-compatible function calling. https://platform.openai.com/docs/guides/function-calling

[6] slam_toolbox. https://github.com/SteveMacenski/slam_toolbox